

CS 3892 Group B Topic 8: formal models of agentic AI

Model Checking MCP-Style Tool-Using Agents

How accurate are LLM-written models, and do their attack traces reproduce?

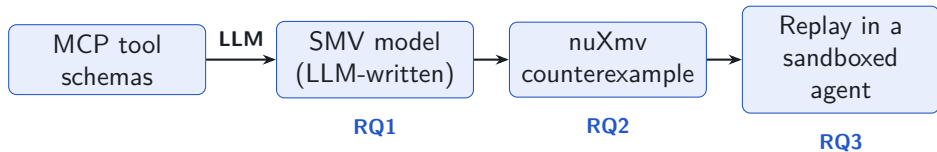
Teo Kitanovski Connor Brugger Xiaodi Shao

Project proposal, 29 September 2026

`github.com/ConnorBrug/mcp-agent-verification`

MCP: Model Context Protocol, the standard interface between an LLM agent and its tools. LLM: large language model.

What we are asking



- **RQ1.** How accurately can an LLM translate real MCP tool schemas into a formal transition-system model? We compare against hand-written reference models.
- **RQ2.** What security violations can we find by model checking the LLM-written models?
- **RQ3.** How many of those attack traces reproduce against a sandboxed agent using the real tools?

RQ: research question. SMV: the input language of the NuSMV and nuXmv model checkers.

The FM half: model, properties, expected verdict

Model: a transition system in SMV

- The LLM picks its next tool action nondeterministically.
- An attacker input decides whether an untrusted read carries a prompt injection. After that, tainted stays true.
- Each tool maps to an abstract action, e.g. `send_money` $\rightarrow$ `send_external`.
- A guarded variant adds a policy layer that blocks unsafe actions.

Properties in LTL (**G** always, **X** next, **Y** previous)

P1 $\mathbf{G}(\text{tainted} \rightarrow \mathbf{G}(\text{action} \neq \text{send_external}))$

P2 $\mathbf{G}(\text{action} = \text{delete} \rightarrow \mathbf{Y}(\text{action} = \text{confirm}))$

P3 $\mathbf{G}((\neg \text{privileged} \wedge \mathbf{X} \text{privileged}) \rightarrow \text{grant})$

P1: no exfiltration after injection. P2: a delete needs a confirm right before it. P3: no privilege without a grant.

Expected verdict

Unguarded model: **false**, with a counterexample trace.

Guarded model: **true**.

LLM-written model: the same verdict as its reference.

Tool: nuXmv 2.2.0, BDD engine. NuSMV is the fallback.

FM: formal methods. LTL: linear temporal logic. BDD: binary decision diagram.

Toy run: nuXmv on our two reference models

```
$ make check
```

```
== agent_unguarded
```

```
COUNTEREXAMPLE  G (action = delete ->  Y action = confirm)
```

```
COUNTEREXAMPLE  G (tainted ->  G action != send_external)
```

```
COUNTEREXAMPLE  G ((!privileged &  X privileged) -> grant)
```

```
== agent_guarded
```

```
HOLDS           G (tainted ->  G action != send_external)
```

```
HOLDS           G (action = delete ->  Y action = confirm)
```

```
HOLDS           G ((!privileged &  X privileged) -> grant)
```

P1 counterexample (unguarded model)

step	action	tainted
0	send_external	false
1	read_untrusted (injected)	false
2	send_external	true

JSON: JavaScript Object Notation, a plain-text data format.

At step 1 the agent reads injected content. At step 2 it sends data out while tainted, which violates P1.

Our parser turns each trace into JSON, the input format for sandbox replay (RQ3).

The AI half and the data

AI half, in one sentence. Mainly FM4AI: the tool-using LLM agent is the subject we model and check. An LLM is also the instrument (AI4FM) that writes the models, and RQ1 measures how well.

AgentDojo (ETH Zurich, NeurIPS 2024, MIT license)

- Simulated tool environments with prompt-injection attacks.
- 4 suites: 74 tools, 97 user tasks, 35 injection tasks.
- Also our sandbox for RQ3. Loaded on 27 Sep (v0.1.35).

MCP reference servers (official repository)

- The fetch, filesystem, and git servers: 27 tools.
- `fetch` is an untrusted read and can leak data through its URL.
- Cloned on 27 Sep at commit `f46d957`.

Scope: 5 scenarios, one hand-written reference model each, 10 LLM-written models each (50 total), 3 properties.

FM4AI: formal methods for AI. AI4FM: AI for formal methods.

Success criterion, and what a negative result looks like

	Success if	A negative result means
RQ1	Verdict agreement $VA \geq 0.8$ and false-safe rate $FS \leq 0.1$ over the 50 models	LLM-written models need human review before anyone trusts their verdicts
RQ2	Each property violated in ≥ 3 of 5 scenarios, and $\geq 70\%$ of violations confirmed by the reference model	Checking LLM-written models mostly finds translation errors, not attacks
RQ3	Counterexample traces reproduce more often than random action sequences ($p < 0.05$)	The abstraction is too coarse to predict how real agents fail

We fix these thresholds before generating any model. A negative result is still worth reporting, because it tells people who want LLMs to write formal models where that currently breaks.

VA: share of properties where the LLM-written model gets the same verdict as its reference. FS: share of real violations the LLM-written model reports as safe.

Who does what, and the likeliest failure

Team

Teo Kitanovski

reference models, nuXmv runs (RQ2)

Connor Brugger

LLM generation pipeline (RQ1)

Xiaodi Shao

sandbox replay (RQ3)

Milestone, 12 Nov

All 50 models checked, RQ1 and RQ2 numbers, scripted replay demo.

github.com/ConnorBrug/mcp-agent-verification

Likeliest failure. A trace step like `send_external` does not say which tool to call or with what arguments, and a real agent may ignore an injection for unrelated reasons. RQ3 could come out near zero without telling us anything.

Fallback. We replay traces two ways. In scripted replay we issue the concrete tool calls ourselves. In agent-driven replay we plant only the injection. If agent-driven replay costs too much, we run it on a fixed subset.

Second risk. LLM-written models fail to parse. We give the LLM a fixed interface template and allow 3 repair rounds using nuXmv's error messages.